

Task 1 - Implement a self-healing script to restart failed containers in a Kubernetes cluster.

Step-by-Step Guide: Self-Healing to Kubernetes (EKS) with Terraform

Goal:

- Deploy an **EKS cluster** with **self-healing features** using **Kubernetes liveness and readiness probes**.
- If a pod crashes, Kubernetes **automatically restarts it**.
- If a node becomes unhealthy, **EKS replaces it automatically** using **Auto Scaling Groups (ASG)**.

Make sure you have connected from your local system to aws account using aws configure command and provide access key id and secret key. (navigate to IAM > user > create new access key) and paste it in Vi ~/.aws/credentials.

Install Required Tools

```
sudo apt update -y
sudo apt install -y unzip
curl -fsSL https://releases.hashicorp.com/terraform/1.7.0/terraform_1.7.0_linux_amd64.zip -o
terraform.zip
unzip terraform.zip
sudo mv terraform /usr/local/bin/
terraform -version
```

```
# Install AWS CLI
sudo apt install awscli -y
aws configure
```

```
# Install kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
kubectl version --client
```

```
# Install eksctl (Optional)
curl -sSL
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz" | tar -
xz
sudo mv eksctl /usr/local/bin/
```

eksctl version

Step to follow:

Step 1 > Make sure you have connected using AWS Access KEY from your Local Machine to AWS Account.

Step2 > create new folder where we will save all config file.

```
mkdir self-healing-script
```

```
cd self-healing-script
```

```
vi main.tf (copy below config file to main.tf and save)
```

```
.....
```

```
provider "aws" {  
  region = "us-east-2" # Change to your AWS region  
}
```

```
# ◇ Create VPC for EKS
```

```
module "vpc" {  
  source = "terraform-aws-modules/vpc/aws"  
  name   = "eks-vpc"  
  cidr   = "10.0.0.0/16"  
  azs    = ["us-east-2a", "us-east-2b"]  
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]  
  private_subnets = ["10.0.3.0/24", "10.0.4.0/24"]  
  enable_nat_gateway = true  
}
```

```
# ◇ Create IAM Role for EKS
```

```
resource "aws_iam_role" "eks_role" {  
  name = "eks-cluster-role"
```

```
  assume_role_policy = jsonencode({  
    Version = "2012-10-17"  
    Statement = [{  
      Action = "sts:AssumeRole"  
      Effect = "Allow"  
      Principal = {  
        Service = ["eks.amazonaws.com", "ec2.amazonaws.com"]  
      }  
    }]  
  })  
}
```

```

resource "aws_iam_role_policy_attachment" "eks_policy" {
  role      = aws_iam_role.eks_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}
resource "aws_iam_role_policy_attachment" "eks_worker_node" {
  role      = aws_iam_role.eks_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}

resource "aws_iam_role_policy_attachment" "eks_cni_policy" {
  role      = aws_iam_role.eks_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
}

resource "aws_iam_role_policy_attachment" "eks_ec2_policy" {
  role      = aws_iam_role.eks_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

# ◇ Create EKS Cluster
resource "aws_eks_cluster" "eks" {
  name      = "self-healing-eks"
  role_arn  = aws_iam_role.eks_role.arn
  vpc_config {
    subnet_ids = module.vpc.public_subnets
  }
}

# ◇ Create Node Group (Auto Scaling)
resource "aws_eks_node_group" "self_healing_nodes" {
  cluster_name = aws_eks_cluster.eks.name
  node_group_name = "self-healing-node-group"
  node_role_arn = aws_iam_role.eks_role.arn
  subnet_ids    = module.vpc.private_subnets
  instance_types = ["t3.medium"]
  scaling_config {
    desired_size = 2
    max_size     = 3
    min_size     = 1
  }
}

```

Deploy Terraform Configuration

```

terraform init
terraform plan

```

```
terraform validate
terraform apply --auto-approve
```

Configure kubectl to Connect to EKS

```
aws eks update-kubeconfig --name self-healing-eks --region
us-east-2
```

```
kubectl get nodes
```

```
ashishsharma@Ashishs-MacBook-Air autoheal % aws eks update-kubeconfig --name self-healing-eks --region us-east-2
Updated context arn:aws:eks:us-east-2:333614583025:cluster/self-healing-eks in /Users/ashishsharma/.kube/config
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-10-0-3-64.us-east-2.compute.inte Ready     <none>   3m11s v1.31.4-eks-aec579
ip-10-0-4-150.us-east-2.compute.int Ready     <none>   3m11s v1.31.4-eks-aec579
ashishsharma@Ashishs-MacBook-Air autoheal % ls
healing1.pem      main.tf           terraform.tfstate      terraform.tfstate.backup
ashishsharma@Ashishs-MacBook-Air autoheal % vi self-healing-app.yaml
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl apply -f self-healing-app.yaml
deployment.apps/self-healing-app created
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get pods -w
NAME                                READY    STATUS    RESTARTS   AGE
self-healing-app-55f75564c4-4lmwt  1/1      Running   0           11s
self-healing-app-55f75564c4-zf5q8  1/1      Running   0           11s
^C
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl delete pod self-healing-app-55f75564c4-4lmwt
pod "self-healing-app-55f75564c4-4lmwt" deleted
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get pods -w
NAME                                READY    STATUS    RESTARTS   AGE
self-healing-app-55f75564c4-qw6w5  0/1      Running   0            5s
self-healing-app-55f75564c4-zf5q8  1/1      Running   0           62s
self-healing-app-55f75564c4-qw6w5  1/1      Running   0            6s
```

Deploy a Self-Healing Kubernetes Application:

We will now deploy an application with liveness and readiness probes to enable self-healing.
Create a Deployment File (self-healing-app.yaml)

.....

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: self-healing-app
labels:
  app: self-healing
spec:
  replicas: 2
  selector:
```



```

matchLabels:
  app: self-healing
template:
  metadata:
    labels:
      app: self-healing
  spec:
    containers:
      - name: self-healing-container
        image: nginx
        ports:
          - containerPort: 80
        resources:
          requests:
            cpu: "100m"
          limits:
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 10
          periodSeconds: 5
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 3

```

Deploy the Application to EKS :

`kubectl apply -f self-healing-app.yaml`

`kubectl get pods -w`

```

ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
self-healing-app-55f75564c4-qw6w5  1/1     Running   0           22m
self-healing-app-55f75564c4-zf5q8  1/1     Running   0           23m
^C
ashishsharma@Ashishs-MacBook-Air autoheal %

```

Test the Self-Healing System

Manually Kill a Pod

kubectl delete pod <POD_NAME>

✓ Kubernetes will automatically restart the pod.

```
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl delete pod self-healing-app-55f75564c4-zf5q8
pod "self-healing-app-55f75564c4-zf5q8" deleted
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get pods -w
NAME                                READY    STATUS    RESTARTS   AGE
self-healing-app-55f75564c4-lsh2f   0/1      Running   0           6s
self-healing-app-55f75564c4-qw6w5   1/1      Running   0          23m
self-healing-app-55f75564c4-lsh2f   1/1      Running   0           7s
^C
ashishsharma@Ashishs-MacBook-Air autoheal %
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get pods -w
NAME                                READY    STATUS    RESTARTS   AGE
self-healing-app-55f75564c4-lsh2f   1/1      Running   0          49s
self-healing-app-55f75564c4-qw6w5   1/1      Running   0          24m
^C
```

kubectl get pods -w

You can run below command to get all cluster and deployment details.

Check Cluster Status - **kubectl cluster-info**

Get Cluster Nodes - **kubectl get nodes -o wide**

Get All Kubernetes Resources in the Cluster - **kubectl get all --all-namespaces**

```
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl cluster-info
Kubernetes control plane is running at https://62617f641472c7d7f309b63f6a61634.sk1.us-east-2.eks.amazonaws.com
CoreDNS is running at https://62617f641472c7d7f309b63f6a61634.sk1.us-east-2.eks.amazonaws.com/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE    VERSION    INTERNAL-IP    EXTERNAL-IP    OS-IMAGE             KERNEL-VERSION    CONTAINER-RUNTIME
ip-10-0-3-44.us-east-2.compute.internal Ready    <none>    34m    v1.31.4-eks-0a8c579  10.0.3.44      <none>          Amazon Linux 2023.6.20250128 6.1.124-134.200.amzn2023.x86_64 containerd://1.7.25
ip-10-0-4-150.us-east-2.compute.internal Ready    <none>    34m    v1.31.4-eks-0a8c579  10.0.4.150     <none>          Amazon Linux 2023.6.20250128 6.1.124-134.200.amzn2023.x86_64 containerd://1.7.25
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get all --all-namespaces
NAMESPACE   NAME                                READY    STATUS    RESTARTS   AGE
default     pod/self-healing-app-55f75564c4-lsh2f 1/1      Running   0           5m57s
default     pod/self-healing-app-55f75564c4-qw6w5 1/1      Running   0           29m
kube-system pod/awx-node-7f4qj                  2/2      Running   0           34m
kube-system pod/awx-node-rtkiv                  2/2      Running   0           34m
kube-system pod/coredns-6b94694fcb-nmj7x    1/1      Running   0           43m
kube-system pod/coredns-6b94694fcb-zq4wn    1/1      Running   0           38m
kube-system pod/kube-proxy-ktb62          1/1      Running   0           34m
kube-system pod/kube-proxy-sk48j          1/1      Running   0           34m

NAMESPACE   NAME                                TYPE    CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
default     service/kubernetes                  ClusterIP  172.20.0.1     <none>          443/TCP          69m
kube-system service/eks-extension-metrics-api    ClusterIP  172.20.143.159 <none>          443/TCP          69m
kube-system service/kube-dns                   ClusterIP  172.20.0.10    <none>          53/UDP,53/TCP,9153/TCP 58m

NAMESPACE   NAME                                DESIRED    CURRENT    READY    UP-TO-DATE    AVAILABLE    NODE SELECTOR    AGE
kube-system daemonset.apps/awx-node    2          2          2          2              2            <none>           58m
kube-system daemonset.apps/kube-proxy 2          2          2          2              2            <none>           58m

NAMESPACE   NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
default     deployment.apps/self-healing-app    2/2      2              2            38m
kube-system deployment.apps/coredns 2/2      2              2            58m

NAMESPACE   NAME                                DESIRED    CURRENT    READY    AGE
default     replicaset.apps/self-healing-app-55f75564c4 2          2          2          39m
kube-system replicaset.apps/coredns-6b94694fcb 2          2          2          59m
ashishsharma@Ashishs-MacBook-Air autoheal %
```

NOTE: to progress with task 2 Don't destroy the infrastructure.

Destroy the infrastructure you have created.

terraform-destroy-auto-approve

Summary

- Terraform-based EKS setup with Auto Scaling
- Kubernetes liveness & readiness probes for pod self-healing
- Node replacement using Auto Scaling when failures occur

Or to continue with Task 2 please keep process with Task 1

Task 2 - Automate Scaling of EKS Nodes Using Prometheus Alerting Rules & Kubernetes HPA

We will **automate scaling** of EKS worker nodes based on **CPU usage** using:

Prometheus Alerting Rules (to detect high CPU usage)

Kubernetes Horizontal Pod Autoscaler (HPA) (to scale pods automatically)

KEDA (Kubernetes Event-Driven Autoscaling) (for event-based scaling)

How it Works?

- KEDA watches Prometheus metrics like `container_cpu_usage_seconds_total`.
- If CPU usage exceeds **50%**, KEDA **triggers scaling up**.
- When CPU drops, it **automatically scales down**.

Install Helm on Your System:

For Ubuntu/Linux

```
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |  
bash
```

For Windows - `choco install kubernetes-helm`

Verify Helm Installation - `helm version`

Install Prometheus & Alert manager on EKS

We will deploy Prometheus using the **Prometheus Operator**.

Add Prometheus Helm Repo & Install Prometheus


```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
helm repo update
```

```
helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace
```

#This should install Prometheus, Alertmanager

If the installation is **corrupted**, uninstall it and retry:

```
helm uninstall prometheus -n monitoring  
kubectl delete namespace monitoring --wait
```

If anything still exists, **force delete** it:

```
kubectl delete all --all -n monitoring  
kubectl delete crds --selector=app.kubernetes.io/name=prometheus-operator
```

Once installation completes, verify everything:

```
kubectl get pods -n monitoring  
kubectl get prometheusrules -n monitoring  
kubectl logs -l app.kubernetes.io/name=prometheus-operator -n monitoring
```

Verify CRDs are Installed

Check if the CRDs for Prometheus are installed:

```
kubectl get crds | grep prometheus
```

You should see output like:

```
alertmanagers.monitoring.coreos.com  
podmonitors.monitoring.coreos.com  
prometheuses.monitoring.coreos.com  
prometheusrules.monitoring.coreos.com  
servicemonitors.monitoring.coreos.com
```

```

metrics-server 1/1 1 0s
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get apiservice | grep metrics
v1.metrics.eks.amazonaws.com kube-system/eks-extension-metrics-api True 94m
v1beta1.external.metrics.k8s.io keda/keda-operator-metrics-apiserver True 78m
v1beta1.metrics.k8s.io kube-system/metrics-server True 64m
ashishsharma@Ashishs-MacBook-Air autoheal %

```

Install KEDA using Helm

`helm repo add kedacore https://kedacore.github.io/charts`

`helm repo update`

`helm install keda kedacore/keda --namespace keda --create-namespace`

KEDA is now installed.

Verify Metrics Server is Running

`kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml`

Check if the metrics server is installed and running:

`kubectl get deployment -n kube-system | grep metrics-server`

Then verify metrics collection:

`kubectl top nodes`

`kubectl top pods -n default`

Define Prometheus Alerting Rule for High CPU

Create a Prometheus alerting rule to **detect high CPU usage**.

Create **cpu-alerts.yaml** and add below config and save.

apiVersion: `monitoring.coreos.com/v1`

kind: `PrometheusRule`

metadata:

name: `high-cpu-alert`

namespace: `monitoring`

```
spec:
  groups:
  - name: cpu-alerts
    rules:
    - alert: HighCPUUsage
      expr: avg(node_cpu_seconds_total{mode!="idle"}) by (instance) > 50
      for: 1m
      labels:
        severity: warning
      annotations:
        summary: "High CPU Usage Detected"
        description: "Node {{ $labels.instance }} has high CPU usage (>50%)"
```

This rule triggers if CPU usage >50% for 1 minute.

Apply the Alert Rule

```
kubectl apply -f cpu-alerts.yaml
```

The alert will be **activated in Prometheus**.

Check if the PrometheusRule is created successfully:

```
kubectl get prometheusrules -n monitoring
```

View logs to confirm Prometheus is running:

```
kubectl logs -l app.kubernetes.io/name=prometheus -n monitoring
```

Check if Prometheus Operator is Running:

```
kubectl get pods -n monitoring
```

Verify PrometheusRule API Version

```
kubectl api-resources | grep prometheus
```

.....

[Restart Prometheus Operator

If you face any issue restart the Prometheus Operator:

```
kubectl rollout restart deployment -n monitoring
```

Check Logs for Errors

If PrometheusRule is **still not working**, check the logs:

```
kubectl logs -l app.kubernetes.io/name=prometheus-operator -n monitoring ]
```

Configure Alertmanager to Trigger Scaling

We will configure **Alertmanager** to send an alert when CPU is high.

Create **alertmanager-config.yaml**

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: alertmanager-config
  namespace: monitoring
data:
  alertmanager.yaml: |
    route:
      receiver: "scale-up"
    receivers:
      - name: "scale-up"
        webhook_configs:
          - url: "http://keda-webhook-service.keda:8080"
```

This **sends alerts to KEDA** to trigger auto-scaling.

Apply the ConfigMap

```
kubectl apply -f alertmanager-config.yaml
```

Create Alerting Rule for Pod Failures

Now, create a **rule for detecting pod crashes**.

Create **pod-failure-alert.yaml**:

```
apiVersion: monitoring.coreos.com/v1
```

```
kind: PrometheusRule
```


metadata:

name: pod-failure-alert

namespace: monitoring

spec:

groups:

- **name:** pod-failure-alerts

rules:

- **alert:** PodCrashLooping

expr: kube_pod_container_status_restarts_total > 3

for: 2m

labels:

severity: critical

annotations:

summary: "Pod is crashing frequently"

description: "Pod {{ \$labels.pod }} in namespace {{ \$labels.namespace }}
is restarting frequently."

Apply the rule:

```
kubectl apply -f pod-failure-alert.yaml
```

Deploy Kubernetes HPA (Horizontal Pod Autoscaler)

We will **auto-scale a deployment** based on **CPU usage**.

Horizontal Pod Autoscaler YAML (**vi hpa.yaml**)

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: self-healing-app

namespace: default

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: self-healing-app

minReplicas: 1

```
maxReplicas: 3
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 50 # Scale up when CPU exceeds 50%
```

Apply the Configuration

```
kubectl apply -f hpa.yaml
```

Verify CPU Metrics

Check if the HPA is detecting CPU usage:

```
kubectl get hpa
```

```
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get hpa
NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
self-healing-app    Deployment/self-healing-app  cpu: 100%/50%    1         5         5          55m
ashishsharma@Ashishs-MacBook-Air autoheal %
```

```
kubectl describe hpa self-healing-app
```

```
kubectl top pods
```

Create HPA for Scaling Pods

```
kubectl autoscale deployment self-healing-app --cpu-percent=50 --min=1 --max=3
```

This scales pods from 1 to 3 when CPU exceeds 50% usage.

Deploy KEDA for Event-Based Scaling

KEDA allows scaling based on Prometheus alerts.

Create a ScaledObject for Scaling Nodes

```
vi cpu-scaledobject.yaml
```

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: cpu-scaledobject
  namespace: monitoring
spec:
  scaleTargetRef:
    kind: Deployment
    name: self-healing-app
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus-server.monitoring.svc.cluster.local
        query: avg(node_cpu_seconds_total{mode!="idle"}) by (instance) > 50
        threshold: "1"
```

This scales **pods** when Prometheus detects **CPU > 50%**

Apply the Scaling Object

```
kubectl apply -f cpu-scaledobject.yaml
```

KEDA will scale pods automatically when CPU usage is high.

```
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get prometheusrule -n monitoring
kubectl logs -l app=alertmanager -n monitoring

NAME                                                    AGE
high-cpu-alert                                          70m
pod-failure-alert                                      40m
prometheus-kube-prometheus-alertmanager.rules          73m
prometheus-kube-prometheus-config-reloaders            73m
prometheus-kube-prometheus-etcd                       73m
prometheus-kube-prometheus-general.rules              73m
prometheus-kube-prometheus-k8s.rules.container-cpu-usage-second 73m
prometheus-kube-prometheus-k8s.rules.container-memory-cache 73m
prometheus-kube-prometheus-k8s.rules.container-memory-rss    73m
prometheus-kube-prometheus-k8s.rules.container-memory-swap   73m
prometheus-kube-prometheus-k8s.rules.container-memory-working-s 73m
prometheus-kube-prometheus-k8s.rules.container-resource      73m
prometheus-kube-prometheus-k8s.rules.pod-owner            73m
prometheus-kube-prometheus-kube-apiserver-availability.rules 73m
prometheus-kube-prometheus-kube-apiserver-burnrate.rules    73m
prometheus-kube-prometheus-kube-apiserver-histogram.rules   73m
prometheus-kube-prometheus-kube-apiserver-slos             73m
prometheus-kube-prometheus-kube-prometheus-general.rules    73m
prometheus-kube-prometheus-kube-prometheus-node-recording.rules 73m
prometheus-kube-prometheus-kube-scheduler.rules           73m
prometheus-kube-prometheus-kube-state-metrics            73m
prometheus-kube-prometheus-kubelet.rules                 73m
prometheus-kube-prometheus-kubernetes-apps               73m
prometheus-kube-prometheus-kubernetes-resources          73m
prometheus-kube-prometheus-kubernetes-storage            73m
prometheus-kube-prometheus-kubernetes-system             73m
prometheus-kube-prometheus-kubernetes-system-apiserver    73m
prometheus-kube-prometheus-kubernetes-system-controller-manager 73m
prometheus-kube-prometheus-kubernetes-system-kube-proxy   73m
prometheus-kube-prometheus-kubernetes-system-kubelet      73m
prometheus-kube-prometheus-kubernetes-system-scheduler    73m
prometheus-kube-prometheus-node-exporter                 73m
prometheus-kube-prometheus-node-exporter.rules           73m
prometheus-kube-prometheus-node-network                  73m
prometheus-kube-prometheus-node.rules                    73m
prometheus-kube-prometheus-prometheus                   73m
prometheus-kube-prometheus-prometheus-operator           73m
No resources found in monitoring namespace.
ashishsharma@Ashishs-MacBook-Air autoheal %
```

Verify Self-Healing is Enabled:

```
kubectl get deployment self-healing-app -o yaml | grep restartPolicy
```

Expected Output:

```
restartPolicy: Always
```

Check Existing Alerts


```
kubectl get prometheusrules -n monitoring
```

This will list all Prometheus alerting rules.

To check if scaling happens after simulating high CPU usage, follow these steps:

Step 1: Verify Alerts & Self-Healing

Test CPU Spike:

Run Stress Test on a Pod

If you already have a `stress-test` pod, restart it:

```
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
self-healing-app-5c7f7fc554-c8dvq   1/1     Running   0           37m
self-healing-app-5c7f7fc554-cqk8d   1/1     Running   0           56m
self-healing-app-5c7f7fc554-dljnz   1/1     Running   0           43m
self-healing-app-5c7f7fc554-klwbs   1/1     Running   0           45m
self-healing-app-5c7f7fc554-wf25w   1/1     Running   0           45m
stress-test                          1/1     Running   0           25s
ashishsharma@Ashishs-MacBook-Air autoheal %
```

kubectl delete pod stress-test

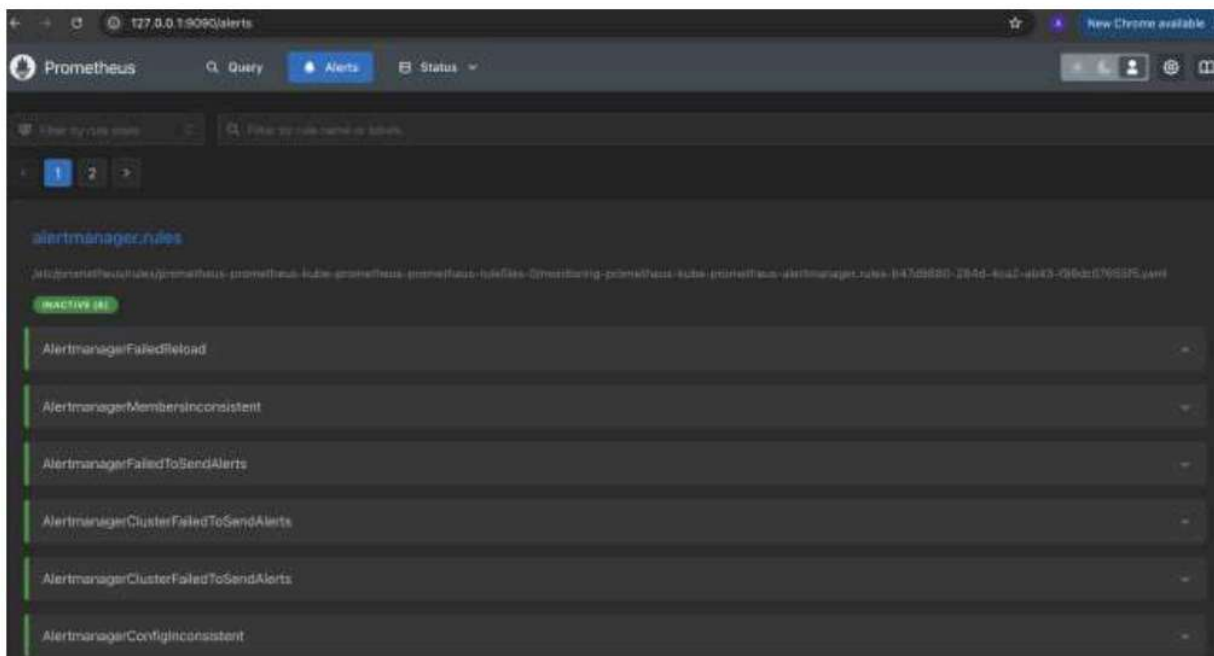
kubectl run stress-test --image=ubuntu --restart=Never -- /bin/bash -c "apt update && apt install -y stress && stress --cpu 2 --timeout 300"

To check the current active alerts, run:

kubectl port-forward -n monitoring prometheus-prometheus-kube-prometheus-prometheus-0 9090

```
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl port-forward -n monitoring prometheus-prometheus-kube-prometheus-prometheus-0 9090
Forwarding from 127.0.0.1:9090 -> 9090
Forwarding from [::1]:9090 -> 9090
Handling connection for 9090
Handling connection for 9090
Handling connection for 9090
Handling connection for 9090
```

<http://127.0.0.1:9090/alerts> open in browser



👉 **Status** → **Alerts** (Check if any alerts are firing)
OR

Step 2: Simulate Pod Failure

To **simulate a pod failure**, manually delete one of the running pods:

```
kubectl get pods
kubectl delete pod <pod-name>
```

(Select anyone from above except stress pod)

Check Alert Status:

```
kubectl get prometheusrule -n monitoring
kubectl logs -l app=alertmanager -n monitoring
```

Step 3: Auto-Healing with Kubernetes

Once an alert is triggered, **Kubernetes automatically restarts failing pods** based on the `restartPolicy: Always`.

Verify Self-Healing:

```
kubectl get pods -n monitoring
(All pod should be running to make sure pods auto start)
```

```
ashishsharma@Ashishs-MacBook-Air autoheal % kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-prometheus-kube-prometheus-alertmanager-0  2/2     Running   0           74m
prometheus-grafana-66b47dbc84-qbmws  3/3     Running   0           74m
prometheus-kube-prometheus-operator-c97f8448f-p7qdk     1/1     Running   0           74m
prometheus-kube-state-metrics-65f864fd45-7bt2r          1/1     Running   0           74m
prometheus-prometheus-kube-prometheus-prometheus-0     2/2     Running   0           74m
prometheus-prometheus-node-exporter-fppct               1/1     Running   0           74m
prometheus-prometheus-node-exporter-pz8hr               1/1     Running   0           74m
ashishsharma@Ashishs-MacBook-Air autoheal %
```

Summary

Prometheus Alerting Rules (to detect high CPU usage)

Kubernetes Horizontal Pod Autoscaler (HPA) (to scale pods automatically)

KEDA (Kubernetes Event-Driven Autoscaling) (for event-based scaling)

Make sure you destroy the infrastructure you have created.

terraform-destroy-auto-approve

.....

Key Differences: HPA vs KEDA

Feature	HPA	KEDA
Scaling Based On	CPU, Memory	Events, HTTP Requests, Queues, Prometheus Metrics
Best For	Predictable workloads	Spiky/Custom workloads
Event-Driven?	✗ No	✓ Yes
Requires Metrics Server?	✓ Yes	✗ No
Integration with Prometheus?	✓ Yes	✓ Yes

We have integrated both in our lab for scenario.